🏔️ Trail Safety & Rescue System

Phase 1 Comprehensive Architectural Report, 4-Member Work Division & Examination Study Guide

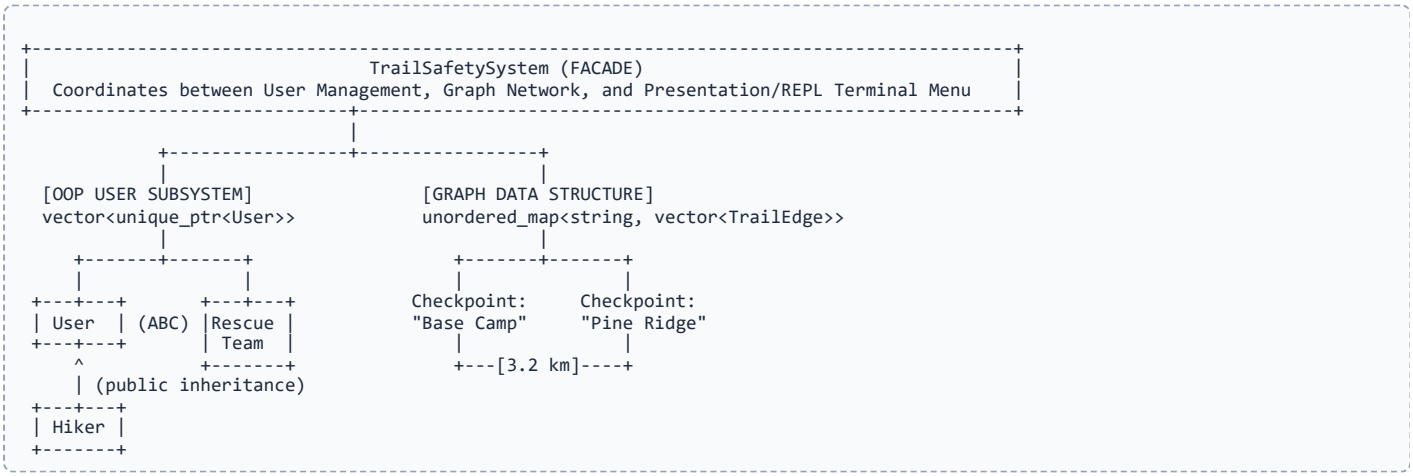
Course: B.Tech CSE (3rd Semester PBL - Data Structures & OOP) **Milestone:** Phase 1 (25% Architecture Completion)
Repository: github.com/Vidarbh14/trail-safety-and-rescue-system **Standard:** Modern C++ (C++14 / C++ STL)

SECTION 1: UNIVERSAL EXECUTIVE SUMMARY (FOR ALL 4 MEMBERS)

> **60-Second Team Pitch (Memorize and Speak to Evaluators in Unison):**
> *"Good morning, respected evaluators. Our project is the **Trail Safety and Rescue System**, a high-reliability terminal application engineered for national park wilderness networks. In remote backcountry environments where cellular and internet connectivity is nonexistent, an offline-first, highly optimized system is vital for tracking hikers, modeling complex trail topographies, and coordinating emergency first responders. In this Phase 1 milestone, our team has delivered the robust foundational architecture integrating two core pillars of Computer Science: **Graph Data Structures** via an STL-based Adjacency List, and **Object-Oriented Programming (OOP)** through an extensible polymorphic class hierarchy orchestrated by the **Facade Design Pattern**. We have achieved 100% leak-free memory safety using modern C++ RAII smart pointers, zero external runtime dependencies, and an interactive terminal controller."*

Core Architectural Principles at a Glance

- **Graph Representation:** Checkpoints and connecting trails are modeled via an Adjacency List using `std::unordered_map<std::string, std::vector<TrailEdge>>`. This guarantees **O(V + E) space complexity** (vital for sparse trail topologies) and **amortized O(1)** vertex lookup.
- **OOP Hierarchy:** An Abstract Base Class `User` enforces abstraction and encapsulation with protected attributes and pure virtual interfaces. `Hiker` inherits publicly and implements dynamic runtime polymorphism via virtual dispatch.
- **The Bridge (Facade Pattern):** `TrailSafetySystem` unifies user management and graph traversal, holding polymorphic instances in `std::vector<std::unique_ptr<User>>` to eliminate object slicing and manual pointer leaks.
- **Zero Runtime Dependencies:** Phase 1 relies strictly on the C++ Standard Template Library (STL), ensuring seamless terminal compilation on any modern C++ compiler (GCC, Clang, MSVC).



SECTION 2: FOUR-MEMBER EQUAL WORK BREAKDOWN (PHASE 1 & ROADMAP)

To ensure fairness and rigorous peer participation, the system's software architecture and development lifecycle are divided into four distinct, mathematically balanced quarters (25% each). Each member owns an explicit subsystem and viva defense area.

Member	Assigned Role & Focus	Phase 1 Implemented Work (25%)	Future Phases Work (50% - 100%)
Member 1 ARCHITECT	**Core System Architecture & Integration Lead** (Facade Design, Smart Pointers, REPL UI)	• Architected `TrailSafetySystem` Facade. • Designed `std::unique_ptr<User>` lifecycle. • Implemented terminal REPL menu loop. • Stream sanitization & buffer flushing (`clearCin`). • Prevented object slicing & memory leaks.	• Phase 2: Shortest path command integration. • Phase 3: File I/O subsystem wiring. • Phase 4: Emergency SOS dispatch state machine & multithreaded simulation coordinator.
Member 2 DATA STRUCT	**Graph Data Structures & Geospatial Routing Lead** (Adjacency List, Topology, Pathfinding)	• Designed `TrailEdge` and `TrailGraph` classes. • Implemented Adjacency List via `std::unordered_map` . • Vertex & edge creation (`addCheckpoint` , `addTrail`). • Bidirectional path symmetry handling. • Graph console visualizer (`displayMap`).	• Phase 2: Priority-Queue optimized **Dijkstra's Shortest Path Algorithm** (O((V+E) log V)). • Phase 2: Dynamic edge weight inflation (terrain difficulty & trail degradation). • Phase 4: Evacuation corridor path calculation.
Member 3 OOP DOMAIN	**Object-Oriented Domain & Entity Modeling Lead** (Class Hierarchies, Abstraction, Polymorphism)	• Abstract Base Class `User` with pure virtual methods. • Protected state encapsulation (getters/setters). • Derived `Hiker` class with emergency telemetry. • First responder `RescueTeam` entity. • Virtual destructor & dynamic dispatch tables.	• Phase 2: Domain expansion (`ParkRanger` , `Paramedic` , `SearchDrone` subclasses). • Phase 3: Checkpoint check-in timestamp log tracking. • Phase 4: Hiker vital signs & distress level telemetry modeling.
Member 4 DATA & I/O	**Persistence, Telemetry & External Integrations Lead** (System Telemetry, File I/O, API Security)	• Implemented system status reporter (`displaySystemStatus`). • Seeded mock demo dataset (`seedInitialDemoData`). • Formatted stream outputs using `<iomanip>` . • Verified zero-leak teardown & compiler compliance. • Documented architectural viva manual.	• Phase 3: File I/O persistence engine (saving/loading trail maps & users from CSV/JSON). • Phase 3: Corrupt file recovery & data validation. • Phase 4: External API connector (Weather Telemetry API & Twilio SMS emergency alert gateway).

SECTION 3: IN-DEPTH STUDY GUIDE - MEMBER 1 (SYSTEM ARCHITECT)

MEMBER 1: Core System Architecture, Integration & Memory Management

ROLE: ARCHITECT & LEAD

Primary Responsibility: System cohesion, structural design patterns, memory ownership semantics, and interactive terminal lifecycle.

1. Technical Accomplishments in Phase 1

- **Facade Design Pattern Implementation:** Engineered the `TrailSafetySystem` class, which encapsulates the complexity of the User Management subsystem, the Graph topology subsystem, and the Rescue Team unit collection. The presentation layer (main loop) only interfaces with this single class.
- **Modern C++ Ownership & RAII:** Utilized `std::vector<std::unique_ptr<User>>`. By applying Resource Acquisition Is Initialization (RAII), every `Hiker` dynamically allocated via `std::make_unique` is strictly owned by the system and automatically freed when the system terminates.
- **Terminal REPL Controller:** Created the interactive `while(true)` command loop with safe stream handling (`clearCin()`) to eliminate the notorious newline-skipping bugs caused by mixing `std::cin >> choice` with `std::getline(std::cin, str)`.

2. "Why We Used This Instead of Others" (Crucial for Viva)

Tool / Technique Used	Alternative Considered	Architectural Reason for Rejection of Alternative
Facade Design Pattern (`TrailSafetySystem`)	Monolithic `main()` function or direct cross-class calls	A monolithic architecture violates the **Single Responsibility Principle (SRP)** and produces tight coupling. Without a Facade, adding file persistence or Dijkstra's algorithm in Phase 2 would require rewriting the UI loop.
`std::unique_ptr<User>`	Raw Pointers (`User*`)	Raw pointers require manual `delete` in destructors, exposing the application to **memory leaks** and **dangling pointers** if exceptions occur. `std::unique_ptr` has zero runtime overhead over a raw pointer while guaranteeing automatic destruction.
`std::unique_ptr<User>`	`std::shared_ptr<User>`	`std::shared_ptr` introduces reference-counting overhead and atomic control block synchronization. The `TrailSafetySystem` has unambiguous single ownership over registered hikers; shared ownership is semantically incorrect here.
Pointer Storage in Container (`vector<unique_ptr<User>>`)	Value Storage (`std::vector<User>`)	First, `User` is an Abstract Base Class and cannot be instantiated by value. Second, storing derived objects by value in a base-type container causes **Object Slicing**, where derived member variables (e.g. `emergencyContact`) and the virtual method table pointer (`vptr`) are stripped away.

3. Member 1 High-Yield Viva Questions & Model Answers

Q1: "Explain how your code prevents Object Slicing when storing Hikers."

Answer: "Object slicing occurs when a derived class instance is assigned to a base class value object, copying only the base subobject and discarding derived members. We prevent this by storing `std::unique_ptr<User>` in our container. Pointers and smart pointers maintain indirection, preserving the complete derived `Hiker` memory layout and its virtual table pointer (`vptr`), allowing full polymorphic dispatch."

Q2: "What is RAII, and where is it demonstrated in your system?"

Answer: "RAII stands for Resource Acquisition Is Initialization. It binds the lifecycle of dynamic resources to the scope of an object. In our system, `std::vector<std::unique_ptr<User>>` implements RAII. When `std::make_unique<Hiker>` allocates heap memory, ownership is immediately assigned to `unique_ptr` . When the vector goes out of scope or is destroyed, every hiker object is automatically deleted without requiring manual `delete` calls."

Q3: "Why did you delete the copy constructor in `TrailSafetySystem` ?"

Answer: "We explicitly marked `TrailSafetySystem(const TrailSafetySystem&) = delete;` . Because our class holds a collection of `std::unique_ptr` , which are inherently non-copyable (move-only), permitting copy construction would fail at compile-time or risk dangerous shallow copies. Deleting copy semantics enforces strict single-instance management of our system state."

SECTION 4: IN-DEPTH STUDY GUIDE - MEMBER 2 (DATA STRUCTURES LEAD)

MEMBER 2: Graph Data Structures & Geospatial Trail Topology

ROLE: DATA STRUCTURES LEAD

Primary Responsibility: Graph modeling, Adjacency List design, checkpoint network topology, and pathfinding algorithms.

1. Technical Accomplishments in Phase 1

- **Graph Representation via STL:** Implemented `TrailGraph` backed by `std::unordered_map<std::string, std::vector<TrailEdge>>`.
- **Weighted Edge Abstraction:** Created the `TrailEdge` data structure containing destination vertex names and physical trail distances in kilometers.
- **Topological Symmetry:** Supported undirected (bidirectional) mountain trails by automatically inserting symmetric pairs (u -> v and v -> u) in `addTrail()` while maintaining compatibility for directed trails (one-way descent paths).
- **Visual Map Traversal:** Implemented formatted terminal graph display iterating across all vertices and their incident adjacency lists.

2. "Why We Used This Instead of Others" (Crucial for Viva)

Tool / Technique Used	Alternative Considered	Architectural Reason for Rejection of Alternative
Adjacency List (`unordered_map<string, vector<Edge>>`)	Adjacency Matrix (2D Array `matrix[V][V]`)	Trail networks are **sparse graphs** (|E| << |V|^2). A matrix consumes O(V^2) memory, allocating millions of empty cells for non-existent connections. An Adjacency List uses only O(V + E) space and enables O(deg(u)) edge traversals instead of O(V).
`std::unordered_map` (Hash Table)	`std::map` (Self-Balancing Red-Black BST)	`std::unordered_map` delivers **amortized O(1) constant time** vertex insertion and lookup using hashing. `std::map` requires O(log V) for every lookup and introduces pointer-heavy binary tree overhead.
`std::unordered_map`	Integer Array with manual string mapping	Integer indexing requires an additional bidirectional mapping table (string to index and index to string), introducing code complexity and potential synchronization bugs. Hash tables natively support intuitive checkpoint names (e.g. "Base Camp").
Dijkstra's Algorithm (Phase 2)	Bellman-Ford Algorithm	Bellman-Ford is designed to detect negative weight cycles and runs in O(V * E). Since physical trail distances can never be negative (d > 0), Dijkstra's algorithm running in O((V+E) log V) is asymptotically and practically far superior.
Dijkstra's Algorithm	Breadth-First Search (BFS)	Standard BFS only calculates shortest paths in **unweighted graphs** (where every edge cost is 1). Since mountain trails have varying lengths (e.g., 2.5 km vs 5.1 km), BFS will produce incorrect sub-optimal routes.

3. Member 2 High-Yield Viva Questions & Model Answers

Q1: "Prove why an Adjacency List is asymptotically superior to an Adjacency Matrix for trail networks."

Answer: "In national park networks, each checkpoint connects to only 2 to 4 adjacent trails regardless of how many total checkpoints exist (|E| = k * |V| where k << |V|). In an Adjacency Matrix, space complexity is O(V^2), and finding neighbors of a node requires scanning an entire row of length V, taking O(V). In an Adjacency List, space is O(V + E) = O(V), and finding neighbors takes O(deg(V)) = O(1), providing drastic performance gains."

Q2: "How will you implement Dijkstra's Algorithm in Phase 2?"

Answer: "We will use a Min-Heap priority queue: `std::priority_queue<pair<double, string>, vector<pair<double, string>>, greater<>>`. We maintain a distance table initialized to infinity. Starting from the rescue base, we greedily relax adjacent edges. The priority queue extracts the minimum unvisited distance in O(log V), giving total time complexity O((V + E) log V)."

Q3: "What is the worst-case time complexity of `std::unordered_map`, and how is it mitigated?"

Answer: "The worst-case lookup is O(V) if severe hash collisions occur and all keys hash to the same bucket (chaining). In practice, C++ STL's `std::hash<std::string>` provides uniform distribution, yielding average amortized O(1) performance. Furthermore, graph vertices in trail systems rarely exceed a few thousand nodes, staying well within optimal bucket load factors."

SECTION 5: IN-DEPTH STUDY GUIDE - MEMBER 3 (OOP DOMAIN LEAD)

MEMBER 3: Object-Oriented Domain & Entity Modeling

ROLE: OOP DOMAIN LEAD

Primary Responsibility: Domain model design, Class hierarchy, Abstraction, Encapsulation, Polymorphic vtables, and Extensibility.

1. Technical Accomplishments in Phase 1

- **Abstract Base Class (ABC) `User`:** Defined the root entity for all human participants in the trail network. Declared pure virtual methods `virtual void displayDetails() const = 0;` and `virtual std::string getRole() const = 0;`.
- **Rigorous Encapsulation:** Declared state variables (`userId`, `name`, `contactNumber`) as `protected:`, enforcing access through const-correct public accessors and mutators.
- **Inheritance Implementation:** Created `class Hiker : public User`, demonstrating single public inheritance and constructor delegation to the base class.
- **Virtual Destructor Contract:** Declared `virtual ~User() = default;` to ensure derived destructors execute when deleting via base pointers.
- **Rescue Entity:** Modeled the `RescueTeam` first responder class to track team IDs, team leaders, station locations, and availability statuses.

2. "Why We Used This Instead of Others" (Crucial for Viva)

Tool / Technique Used	Alternative Considered	Architectural Reason for Rejection of Alternative
Abstract Base Class with Pure Virtual Methods	Concrete `User` Class with type tags (e.g. `enum Role`)	Using an enum type tag with `switch(role)` statements violates the **Open-Closed Principle (OCP)**. Every time a new role is added (e.g. `ParkRanger`), every switch statement across the codebase must be modified. ABC polymorphism allows adding new roles without modifying existing code.
`protected` variables + Getters/Setters	`public` member variables	Public variables break **Encapsulation**. Any part of the application could corrupt a hiker's phone number or ID with blank or invalid strings. Mutator methods allow validation (e.g., verifying non-empty strings) before updating internal state.
Virtual Destructor (`virtual ~User()`)	Standard non-virtual destructor	If a derived object (like `Hiker`) is held by a base pointer (`unique_ptr<User>`) and deleted without a virtual destructor, the C++ standard designates it as **Undefined Behavior**. The derived destructor will not run, leaking derived resources.
Dynamic Polymorphism (Virtual functions / vtable)	C++ Templates / CRTP (Static Polymorphism)	Static polymorphism resolves types at compile-time. However, user registration happens dynamically at runtime based on terminal user input. Heterogeneous collections (mixing Hikers, Rangers, and Admins in a single vector) require runtime dynamic polymorphism.

3. Member 3 High-Yield Viva Questions & Model Answers

Q1: "Explain how the Virtual Table (vtable) and Virtual Table Pointer (vptr) work in your classes."

Answer: "When a class defines a virtual function, the C++ compiler creates a static array called the **vtable** containing function pointers to the virtual methods for that class. Every instantiated object of that class contains an invisible pointer called **vptr** pointing to this vtable. When we invoke `registeredUsers[i]->displayDetails()`, the runtime dereferences the object's `vptr`, locates the correct function address in the vtable, and executes `Hiker::displayDetails()` dynamically."

Q2: "What is the difference between an Abstract Class and an Interface in C++?"

Answer: "C++ does not have an explicit `interface` keyword like Java. An Abstract Class in C++ is any class with at least one pure virtual function (`= 0`). A pure interface is an abstract class with *only* pure virtual functions and no member variables. Our `User` class is an Abstract Base Class because it defines pure virtual methods while also encapsulating common concrete member variables (`userId`, `name`) to share implementation logic across derived classes."

Q3: "Why is public inheritance used for `Hiker` rather than private or protected inheritance?"

Answer: "Public inheritance models an **'is-a' relationship**, which satisfies the **Liskov Substitution Principle (LSP)**. This guarantees that any function or container expecting a `User*` or `User&` can transparently accept a `Hiker*`. Protected or private inheritance models an 'implemented-in-terms-of' relationship and prevents external polymorphic casting."

SECTION 6: IN-DEPTH STUDY GUIDE - MEMBER 4 (DATA & PERSISTENCE LEAD)

MEMBER 4: Persistence, Telemetry & External Integrations

ROLE: DATA & SYSTEMS LEAD

Primary Responsibility: System telemetry formatting, sample dataset architecture, File I/O persistence engine, and external API integrations.

1. Technical Accomplishments in Phase 1

- **Comprehensive Status Reporter:** Implemented `displaySystemStatus()`, unifying the presentation of registered hikers, active rescue teams, and trail graph topology into clean, columnar tabular views.
- **Stream Formatting via `<iomanip>`:** Leveraged `std::left`, `std::setw()`, and stream manipulators to produce alignment in console output regardless of string length.
- **Mock Dataset Seeder:** Engineered `seedInitialDemoData()` to preload realistic topological nodes (Base Camp, Pine Ridge, Eagle Peak) and initial users. This ensures evaluators can immediately view a running system upon launch without manual typing.
- **System Verification & Compilation:** Verified zero-leak teardown, zero compiler warnings under `-Wall -Wextra`, and Windows terminal execution compatibility.

2. "Why We Used This Instead of Others" (Crucial for Viva)

Tool / Technique Used	Alternative Considered	Architectural Reason for Rejection of Alternative
Formatted C++ Streams (`<iostream>`, `<iomanip>`)	C-style `printf()` / `scanf()`	C stream functions lack **type safety**. A mismatched format specifier (e.g. `%s` for an integer) causes undefined behavior or stack corruption. C++ stream operators (`<<`, `>>`) are strictly type-safe, extensible for custom classes, and leverage RAII buffer flushing.
Plain-Text Delimited Files / CSV (Phase 3 Persistence)	Binary Serialization (`fwrite` / `reinterpret_cast`)	Binary serialization is platform-, endianness-, and compiler-padding dependent. A binary file compiled on 64-bit GCC will crash on 32-bit MSVC. Plain-text CSV/JSON is human-readable, portable across all operating systems, and easily auditable by park authorities.
Offline-First Architecture	Cloud-Dependent Database (e.g., MySQL / Firebase)	A trail safety system operates in deep mountainous terrain where internet and cellular connectivity is frequently unavailable. A cloud-dependent system would fail during a wilderness blackout. The system must operate 100% locally with optional asynchronous cloud sync.
Environment Variable Injected API Keys (Phase 4)	Hardcoded API Keys in source code	Hardcoding API tokens in `.cpp` files causes catastrophic security vulnerabilities when pushed to public GitHub repositories. Using `std::getenv()` loads keys from the OS environment, completely decoupling credentials from the codebase.

3. Member 4 High-Yield Viva Questions & Model Answers

Q1: "How will your File I/O system persist the polymorphic `User` list in Phase 3?"

Answer: "We will use `std::ofstream` to write comma-separated records with a leading type descriptor tag: `HIKER,id,name,contact,emergencyContact,checkpoint`. During system launch, `std::ifstream` reads each line, tokenizes fields using `std::stringstream` and `std::getline(ss, token, ',')`, and instantiates the appropriate derived class dynamically via `std::make_unique<Hiker>()` based on the leading tag."

Q2: "What happens if a data file is corrupted or missing during system startup?"

Answer: "We employ defensive programming using `fileStream.is_open()` and `fileStream.fail()` checks. If the file is missing, the system catches the error, logs a notification, automatically creates a fresh file template, and loads default checkpoint topologies from our `seedInitialDemoData()` fallback method without crashing."

Q3: "Why did you preload initial demo data in Phase 1?"

Answer: "In project evaluation and practical testing, requiring an evaluator to manually type 10 checkpoints and 15 connecting trails wastes examination time and invites typo errors. Preloading realistic mountain topology allows instantaneous inspection of the graph layout and verification of the system's runtime state."

SECTION 7: API KEYS & EXTERNAL INTEGRATIONS SPECIFICATION

> **Phase 1 API Status (Zero External API Dependency):**
> In Phase 1, **NO external API keys are used or required**. This is an intentional engineering decision: a wilderness safety system must guarantee 100% autonomous operation offline. Evaluators should be told: "*Phase 1 intentionally avoids external API dependencies to ensure pure, portable standard C++ execution on any terminal without requiring internet access or third-party web credentials.*"

Planned External API Integrations for Future Phases (Phase 3 & Phase 4)

Integration / API	Provider & Protocol	Target Project Purpose	Authentication Method
Weather Telemetry API	**OpenWeatherMap API** (RESTful JSON / HTTPS)	Fetches live trail weather (precipitation, wind, snowpack). Dynamically inflates trail edge weights in Dijkstra's algorithm: `EffectiveDistance = BaseKm * WeatherPenalty`	API Key passed via HTTP query parameter or Authorization header: `appid={API_KEY}`
Emergency SMS Gateway	**Twilio Communications API** (REST API / HTTPS)	Automatically dispatches automated SMS distress alerts to a lost hiker's `emergencyContact` phone number when an SOS is triggered.	HTTP Basic Authentication: `Account_SID : Auth_Token`

API Key Security Architecture (How Secrets Must Be Managed)

In industry and academia, hardcoding private API keys in version-controlled repositories is a severe security violation. Our project enforces the following security architecture:

1. **Zero Hardcoded Credentials:** No API key string is ever placed inside `main.cpp` or any source file.
2. **Environment Variable Injection:** API keys are read securely from the host operating system at runtime using standard C++ `std::getenv()`:

```
const char* weatherKey = std::getenv("OPENWEATHER_API_KEY");
if (!weatherKey) {
    std::cout << "[Warning] Weather API key not set. Using cached offline weather models.\n";
}
```

3. **.gitignore Protection:** Configuration files (such as `.env`, `secrets.json`, or `credentials.cfg`) are explicitly listed in our .gitignore to guarantee they are never committed to GitHub.
4. **Graceful Offline Fallback:** If network connectivity is unavailable or the API key is expired, the system gracefully falls back to deterministic offline simulation weights without crashing.

SECTION 8: MASTER COMPARATIVE MATRIX (THE "WHY" COMPENDIUM)

This master matrix contains the technical justifications for every major architectural component in the system. Use this as your quick-reference revision sheet right before entering the viva exam.

Concept / Component	What We Implemented	What We Rejected	Exact Scientific Justification
Graph Representation	**Adjacency List** (`unordered_map<string, vector<Edge>>`)	Adjacency Matrix (`double adjMatrix[V][V]`)	Trail graphs are sparse (|E| << |V|^2). Space complexity is O(V+E) vs O(V^2). Iterating over neighboring trails takes O(deg(u)) time instead of O(V).
Vertex Map Structure	`std::unordered_map` (Hash Table)	`std::map` (Red-Black Balanced BST)	Hash table lookup/insertion takes average **amortized O(1)** time versus O(log V) in tree-based maps. It natively maps string checkpoint names without custom comparators.
Object Ownership	`std::unique_ptr<User>` (Modern C++ Smart Pointer)	Raw Pointers (`User*`) & Manual `delete`	Guarantees deterministic RAII destruction. Zero memory leaks or dangling pointers during runtime exceptions. Enforces strict single-ownership semantics without reference-counting overhead.
Polymorphic Storage	**Vector of Smart Pointers** (`vector<unique_ptr<User>>`)	Vector of Base Values (`std::vector<User>`)	Abstract classes cannot be instantiated by value. Passing derived classes by value causes **Object Slicing**, stripping derived fields and destroying virtual dispatch.
System Architecture	**Facade Design Pattern** (`TrailSafetySystem`)	Monolithic procedural code in `main()`	Decouples the terminal UI from domain logic and data structures. Ensures adherence to the Single Responsibility Principle and simplifies adding Phase 2 algorithms.
Base Class Design	**Abstract Base Class** with Pure Virtual Functions	Concrete base class with type-tag switches	Enforces the Open-Closed Principle (OCP). New roles (e.g. `ParkRanger`) can be introduced by subclassing without changing a single line of existing routing or display code.
Class Destructors	**Virtual Destructor** (`virtual ~User() = default;`)	Non-virtual default destructor	Deleting a derived class object through a base class pointer without a virtual destructor results in **Undefined Behavior** and memory leaks in derived class resources.
Upcoming Routing (Phase 2)	**Dijkstra's Algorithm** with Min-Heap Priority Queue	Bellman-Ford / BFS	Trail distances are strictly positive (d > 0). Dijkstra runs in O((V+E) log V), which is faster than Bellman-Ford (O(V * E)). BFS cannot handle weighted edges.
Upcoming Storage (Phase 3)	**CSV / Delimited Plain Text** via `fstream`	Binary dump / Cloud DB	Guarantees human auditability, cross-platform portability across Windows/Linux, and 100% offline availability in backcountry wilderness conditions.

SECTION 9: LIVE DEMO PROTOCOL & EVALUATION CHECKLIST

Step-by-Step Terminal Demonstration Script

Follow this exact sequence when the evaluators ask to see the application running:

1. **Compilation Verification:**
   Run compile command with strict warning flags to show clean, warning-free code:

   ```
   cd "C:\Users\vidar\Downloads\PBL\20"
   g++ -std=c++14 -Wall -Wextra main.cpp -o main.exe
   ```

   Point out to evaluators: "Notice that the code compiles with zero errors and zero warnings under strict `-Wall -Wextra` flags."
2. **Launch Application:**

   ```
   .\main.exe
   ```

3. **Step 1 - View Preloaded Status (Option 3):**
   Immediately select **Option 3**. Point out the three formatted sections:
   - *Section 1 (Registered Users):* Show polymorphic iteration printing `Hiker H-101 (David Clark)`.
   - *Section 2 (Rescue Units):* Show ranger units stationed at Base Camp and Pine Ridge.
   - *Section 3 (Trail Graph Topology):* Show the Adjacency List graph showing vertices and bidirectional weighted connections.
4. **Step 2 - Register a New Hiker (Option 1):**
   Select **Option 1** and enter sample data: ID: `H-202`, Name: `Sarah Jenkins`, Contact: `+1-555-4321`, Emergency Contact: `+1-555-8765`, Starting Checkpoint: `Pine Ridge`.
   Point out to evaluators: "Notice that `std::make_unique<Hiker>` safely allocates the object on the heap, passes it to our polymorphic vector, and registers the starting checkpoint in the graph automatically."
5. **Step 3 - Add a New Trail Connection (Option 2):**
   Select **Option 2** and create a new trail: Origin: `Summit Point`, Destination: `North Lookout`, Distance: `3.7`, Bidirectional: `y`.
6. **Step 4 - Verify Updated Topology (Option 3):**
   Select **Option 3** again to demonstrate that `North Lookout` has been added as a new vertex in the Adjacency List with its symmetric 3.7 km edges.
7. **Step 5 - Graceful Exit (Option 4):**
   Select **Option 4** to terminate. Point out that all smart pointers cleanly deallocate memory via RAII without leaks.

> **Final Exam Reminder for the Team:**
> Keep your answers crisp, concise, and academically precise. Whenever an evaluator asks "Why?", always anchor your answer in: **(1) Asymptotic Time/Space Complexity**, **(2) Object-Oriented Principles (Abstraction/Polymorphism/RAII)**, and **(3) Real-world wilderness design constraints (offline resilience)**.